

Nama : Kamila Zahra R.
NPM: 247006111161
Kelas: F
Praktikum 2_MPI

Tugas Praktikum

Message Passing Interface (MPI)

PENDAHULUAN

Modul ini berisi lima tugas praktikum MPI untuk memahami komunikasi dasar antar proses: Scatter, Gather, kombinasi Scatter+Gather, serta Reduce vs Allreduce. Praktikum disusun bertahap dari distribusi data, transformasi lokal, hingga agregasi global. Setiap tugas menyertakan petunjuk eksekusi, starter code, ruang tabel hasil/grafik, dan pertanyaan diskusi & analisis.

Tugas Praktikum 1 — Scatter Sederhana

Tujuan

- Memahami distribusi data dari rank 0 ke seluruh proses menggunakan Scatter.
- Mengamati perilaku urutan output pada eksekusi paralel.

Spesifikasi

1. Proses 0 memiliki list [10, 20, 30, 40].
2. Data dibagi rata ke 4 proses dengan scatter.
3. Setiap proses mencetak data yang diterimanya.

Starter Code

```
# Nama      : Kamila Zahra R.  
# NPM       : 247006111161  
# Kelas     : F  
# Deskripsi: Tugas Praktikum 1 - Scatter Sederhana  
  
from mpi4py import MPI  
  
def main():  
    comm = MPI.COMM_WORLD  
    rank = comm.Get_rank()  
    size = comm.Get_size()
```



```

# Pastikan jumlah proses = 4
if size != 4:
    if rank == 0:
        print("Harap jalankan dengan -n 4")
        raise SystemExit

data = [10, 20, 30, 40] if rank == 0 else None
recv = comm.scatter(data, root=0)

print(f"Rank {rank} menerima {recv}")

if __name__ == "__main__":
    main()

```

Tangkapan Layar

The screenshot shows a code editor with a file named `tugas1.py`. The script is a Python program using `mpi4py` to demonstrate a scatter operation. It checks if the number of processes is 4, and if not, it prints a message and exits. It then defines a `main` function that initializes an MPI communicator, gets the rank and size, and performs a scatter operation on a list of data. The script is executed in a terminal window using the command `python tugas1.py`. The output shows the rank of each process and the data it received.

```

tugas1.py > ...
1  # Nama      : Kamila Zahra R.
2  # NPM       : 247006111161
3  # Kelas     : F
4  # Deskripsi: Tugas Praktikum 1 - Scatter Sederhana
5
6
7  from mpi4py import MPI
8
9  def main():
10     comm = MPI.COMM_WORLD
11     rank = comm.Get_rank()
12     size = comm.Get_size()
13
14     # Pastikan jumlah proses = 4
15     if size != 4:
16         if rank == 0:
17             print("Harap jalankan dengan -n 4")
18             raise SystemExit
19
20     data = [10, 20, 30, 40] if rank == 0 else None
21     recv = comm.scatter(data, root=0)
22
23     print(f"Rank {rank} menerima {recv}")
24
25 if __name__ == "__main__":
26     main()

```

Problems Output Debug Console Terminal Ports

```

PS D:\Praktikum_MPI> & "C:\Program Files\Microsoft MPI\Bin\mpiexec.exe" -n 4 python tugas1.py
Rank 0 menerima 10
Rank 1 menerima 20
Rank 2 menerima 30
Rank 3 menerima 40
PS D:\Praktikum_MPI>

```

Tabel Hasil

Rank	Data Diterima
0	10
1	20
2	30
3	40

Pertanyaan Diskusi

1. Apa output yang dihasilkan tiap proses? (Harap tuliskan hasil observasi)
Output yang dihasilkan tiap proses berupa pesan yang menampilkan ID Rank dan nilai data yang ditangkap setelah fungsi scatter:
 - Rank 0 menerima 10
 - Rank 1 menerima 20
 - Rank 2 menerima 30
 - Rank 3 menerima 40
2. Mengapa urutannya bisa berbeda saat dicetak? (Bahas sifat output paralel & buffering stdout)
Urutan pencetakan di terminal dapat berubah-ubah (acak) karena dalam eksekusi paralel, setiap proses (rank) berjalan secara independen dan asinkron di core CPU tersendiri.
Selain itu, sistem operasi menggunakan mekanisme *buffering stdout* (penampungan sementara teks sebelum ditayangkan ke layar terminal). Kapan dan urutan mana *buffer* tersebut dikosongkan (*flushed*) ke terminal tergantung pada penjadwalan sistem operasi (OS *scheduler*), bukan berdasarkan nomor rank secara berurutan.

Tugas Praktikum 2 — Gather Sederhana

Tujuan

- Memahami pengumpulan data dari semua proses ke root menggunakan **Gather**.
- Mengamati perubahan hasil saat jumlah proses berubah.

Spesifikasi

1. Tiap proses mengirim data rank * 5.
2. Data dikumpulkan di proses 0 dan dicetak sebagai list.

Starter Code

```
# Nama      : Kamila Zahra R.
# NPM       : 247006111161
# Kelas     : F
# Deskripsi: Tugas Praktikum 2 - Gather Sederhana

from mpi4py import MPI

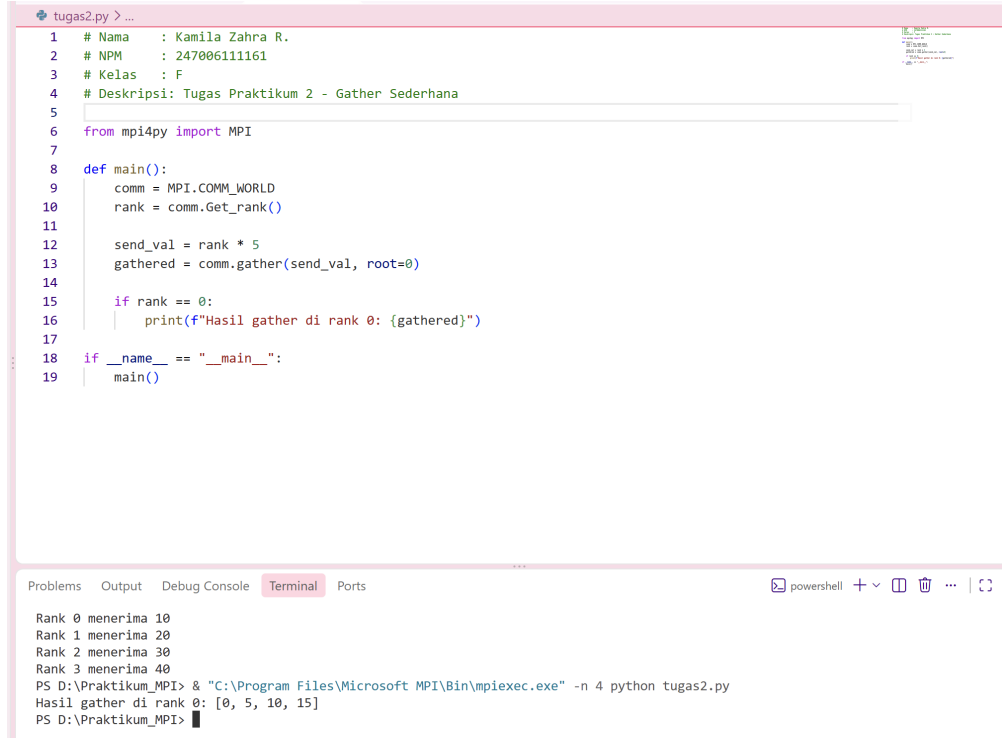
def main():
    comm = MPI.COMM_WORLD
    rank = comm.Get_rank()

    send_val = rank * 5
    gathered = comm.gather(send_val, root=0)

    if rank == 0:
        print(f"Hasil gather di rank 0: {gathered}")

if __name__ == "__main__":
    main()
```

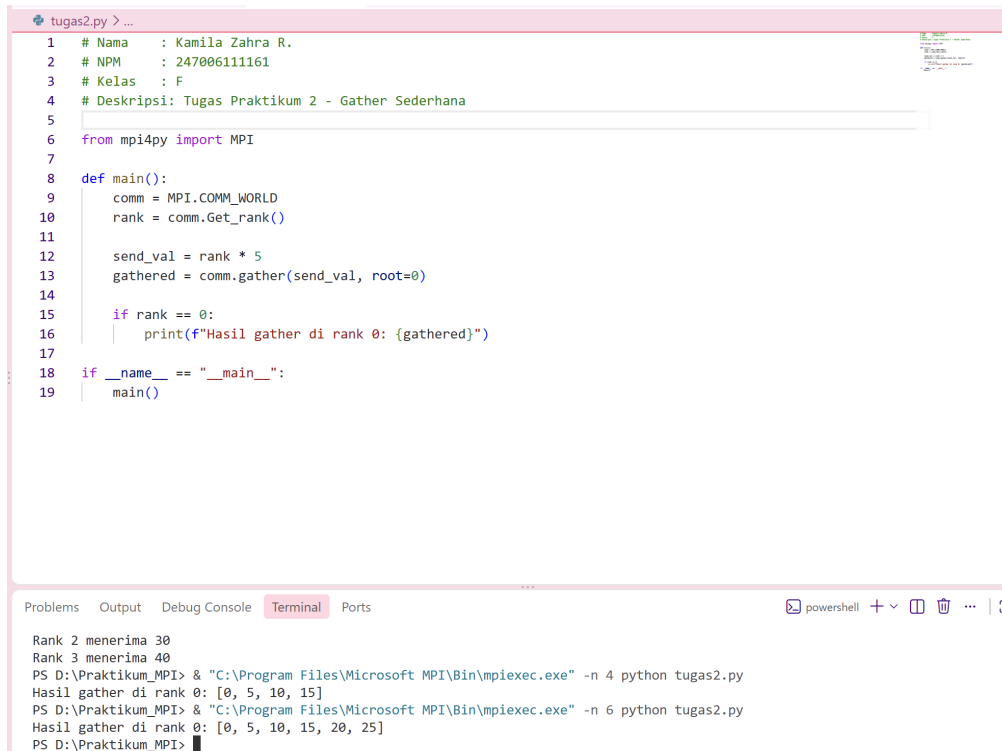
Tangkapan Layar



```
tugas2.py > ...
1 # Nama : Kamila Zahra R.
2 # NPM : 247006111161
3 # Kelas : F
4 # Deskripsi: Tugas Praktikum 2 - Gather Sederhana
5
6 from mpi4py import MPI
7
8 def main():
9     comm = MPI.COMM_WORLD
10    rank = comm.Get_rank()
11
12    send_val = rank * 5
13    gathered = comm.gather(send_val, root=0)
14
15    if rank == 0:
16        print(f"Hasil gather di rank 0: {gathered}")
17
18 if __name__ == "__main__":
19     main()
```

Problems Output Debug Console Terminal Ports

Rank 0 menerima 10
Rank 1 menerima 20
Rank 2 menerima 30
Rank 3 menerima 40
PS D:\Praktikum_MPI> & "C:\Program Files\Microsoft MPI\Bin\mpiexec.exe" -n 4 python tugas2.py
Hasil gather di rank 0: [0, 5, 10, 15]
PS D:\Praktikum_MPI>



```
tugas2.py > ...
1 # Nama : Kamila Zahra R.
2 # NPM : 247006111161
3 # Kelas : F
4 # Deskripsi: Tugas Praktikum 2 - Gather Sederhana
5
6 from mpi4py import MPI
7
8 def main():
9     comm = MPI.COMM_WORLD
10    rank = comm.Get_rank()
11
12    send_val = rank * 5
13    gathered = comm.gather(send_val, root=0)
14
15    if rank == 0:
16        print(f"Hasil gather di rank 0: {gathered}")
17
18 if __name__ == "__main__":
19     main()
```

Problems Output Debug Console Terminal Ports

Rank 2 menerima 30
Rank 3 menerima 40
PS D:\Praktikum_MPI> & "C:\Program Files\Microsoft MPI\Bin\mpiexec.exe" -n 4 python tugas2.py
Hasil gather di rank 0: [0, 5, 10, 15]
PS D:\Praktikum_MPI> & "C:\Program Files\Microsoft MPI\Bin\mpiexec.exe" -n 6 python tugas2.py
Hasil gather di rank 0: [0, 5, 10, 15, 20, 25]
PS D:\Praktikum_MPI>

Tabel Hasil

Proses	Hasil List di Rank 0
4	[0,5,10,15]
6	[0,5,10,15,20,25]

Pertanyaan Diskusi

1. Apa isi list hasil di proses 0 jika ada 4 proses?
Isi list-nya adalah [0, 5, 10, 15]. Hal ini diperoleh dari perhitungan $\text{rank} * 5$ untuk masing-masing rank (0, 1, 2, dan 3).
2. Bagaimana hasilnya jika jumlah proses = 6?
Isi list-nya menjadi [0, 5, 10, 15, 20, 25]. List otomatis bertambah panjang mengikuti jumlah proses ($\text{rank} * 5$ dari rank 0 sampai rank 5).
3. Jelaskan mengapa urutan elemen sesuai rank.
Fungsi `comm.gather()` secara eksplisit mengumpulkan data dari seluruh proses dan menyusunnya kembali ke dalam satu array/list di *root process* (rank 0) **berdasarkan urutan ID Rank pengirimnya** (dari rank 0, 1, 2, dst.), bukan berdasarkan waktu proses mana yang lebih dulu selesai mengirim data.

Tugas Praktikum 3 — Scatter + Gather (Kombinasi)

Tujuan

- Menggabungkan distribusi data (scatter), komputasi lokal, dan pengumpulan hasil (gather).
- Memvalidasi transformasi elemen (kuadrat) per-proses.

Spesifikasi

1. Proses 0 memiliki list [1, 2, 3, 4].
2. Data di-scatter ke 4 proses.
3. Setiap proses menghitung kuadrat dari datanya.
4. Hasil di-gather kembali ke proses 0.

Starter Code

```
# Nama      : Kamila Zahra R.
# NPM       : 247006111161
# Kelas     : F
# Deskripsi: Tugas Praktikum 3 - Scatter + Gather (Kombinasi)

from mpi4py import MPI

def main():
    comm = MPI.COMM_WORLD
    rank = comm.Get_rank()
    size = comm.Get_size()

    if size != 4:
        if rank == 0:
            print("Harap jalankan dengan -n 4")
            raise SystemExit

    # Pengujian default: [1, 2, 3, 4]
    data = [1, 2, 3, 4] if rank == 0 else None
    # Jika ingin menguji [2, 4, 6, 8], ganti variabel data di atas.

    x = comm.scatter(data, root=0)
    y = x * x
    hasil = comm.gather(y, root=0)
```

```

if rank == 0:

    print(f"Input  : {data}")

    print(f"Output : {hasil}")

if __name__ == "__main__":

    main()

```

Tangkapan Layar

```

tugas3.py > ...
1  # Nama      : Kamila Zahra R.
2  # NPM       : 247006111161
3  # Kelas    : F
4  # Deskripsi: Tugas Praktikum 3 - Scatter + Gather (Kombinasi)
5
6  from mpi4py import MPI
7
8  def main():
9      comm = MPI.COMM_WORLD
10     rank = comm.Get_rank()
11     size = comm.Get_size()
12
13     if size != 4:
14         if rank == 0:
15             print("Harap jalankan dengan -n 4")
16             raise SystemExit
17
18     # Pengujian default: [1, 2, 3, 4]
19     data = [1, 2, 3, 4] if rank == 0 else None
20     # Jika ingin menguji [2, 4, 6, 8], ganti variabel data di atas.
21
22     x = comm.scatter(data, root=0)
23     y = x * x
24     hasil = comm.gather(y, root=0)
25
26     if rank == 0:
27         print(f"Input  : {data}")
28         print(f"Output : {hasil}")
29

```

Problems Output Debug Console Terminal Ports

Hasil gather di rank 0: [0, 5, 10, 15]
PS D:\Praktikum_MPI> & "C:\Program Files\Microsoft MPI\Bin\mpiexec.exe" -n 6 python tugas2.py
Hasil gather di rank 0: [0, 5, 10, 15, 20, 25]
PS D:\Praktikum_MPI> & "C:\Program Files\Microsoft MPI\Bin\mpiexec.exe" -n 4 python tugas3.py
Input : [1, 2, 3, 4]
Output : [1, 4, 9, 16]

Tabel Hasil

Rank	Input	Output (Kuadrat)
0	1	1
1	2	4
2	3	9
3	4	16

Pertanyaan Diskusi

1. Apa hasil akhir yang dikumpulkan di proses 0? untuk [1,2,3,4].

Hasil akhirnya berupa list [1, 4, 9, 16]. Setiap elemen merupakan hasil pengkuadratan ($y = X \times X$) dari nilai elemen awal di masing-masing rank.

2. Bagaimana jika list awal diganti [2,4,6,8]?

Hasil akhirnya akan menjadi list [4, 16, 36, 64]. Proses pengkuadratan lokal tetap berlaku ($2^2=4$, $4^2=16$, $6^2=36$, $8^2=64$).

3. Jelaskan korespondensi indeks (rank $\leftrightarrow$ posisi data).

Saat Scatter: Elemen pada indeks ke- i dari list asal di rank 0 didistribusikan langsung ke proses yang memiliki ID Rank = i (misal: indeks 0 ke Rank 0, indeks 1 ke Rank 1, dst.).

Saat Gather: Hasil komputasi lokal dari proses dengan ID Rank = i dikumpulkan kembali dan diletakkan secara presisi pada indeks ke- i dalam list hasil akhir di rank 0. Dengan kata lain, terdapat hubungan satu-ke-satu (1:1) yang konsisten antara indeks elemen dalam list dengan ID Rank proses yang mengolahnya.

Tugas Praktikum 4 — Hitung Jumlah dengan Scatter + Gather (Bonus)

Tujuan

- Menerapkan transformasi lokal dan agregasi total di rank 0.
- Melatih validasi hasil komputasi terdistribusi.

Spesifikasi

1. Proses 0 punya list [5, 10, 15, 20] (4 elemen).
2. Data di-scatter.
3. Tiap proses menghitung $2 \times$ data.
4. Hasil di-gather, lalu dijumlahkan di proses 0.

Starter Code

```
# Nama      : Kamila Zahra R.
# NPM       : 247006111161
# Kelas     : F
# Deskripsi: Tugas Praktikum 4 - Hitung Jumlah dengan Scatter + Gather

from mpi4py import MPI

def main():
    comm = MPI.COMM_WORLD
    rank = comm.Get_rank()
    size = comm.Get_size()

    if size != 4:
        if rank == 0:
            print("Harap jalankan dengan -n 4")
            raise SystemExit

    data = [5, 10, 15, 20] if rank == 0 else None
    x = comm.scatter(data, root=0)
    local = 2 * x
    kumpul = comm.gather(local, root=0)

    if rank == 0:
```

```

total = sum(kumpul)

print(f>Data awal          : {data}")
print(f"*2 per-proses     : {kumpul}")
print(f>Total akhir       : {total}")

if __name__ == "__main__":
    main()

```

Tangkapan Layar

```

tugas4.py > main
1  # Nama      : Kamila Zahra R.
2  # NPM       : 247006111161
3  # Kelas     : F
4  # Deskripsi: Tugas Praktikum 4 - Hitung Jumlah dengan Scatter + Gather
5
6  from mpi4py import MPI
7
8  def main():
9      comm = MPI.COMM_WORLD
10     rank = comm.Get_rank()
11     size = comm.Get_size()
12
13     if size != 4:
14         if rank == 0:
15             print("Harap jalankan dengan -n 4")
16             raise SystemExit
17
18     data = [5, 10, 15, 20] if rank == 0 else None
19     x = comm.scatter(data, root=0)
20     local = 2 * x
21     kumpul = comm.gather(local, root=0)
22
23     if rank == 0:
24         total = sum(kumpul)
25         print(f>Data awal          : {data}")
26         print(f"*2 per-proses     : {kumpul}")
27         print(f>Total akhir       : {total}")
28

```

Terminal Output:

```

Input : [1, 2, 3, 4]
Output : [1, 4, 9, 16]
PS D:\Praktikum_MPI> "C:\Program Files\Microsoft MPI\Bin\mpiexec.exe" -n 4 python tugas4.py
Data awal      : [5, 10, 15, 20]
*2 per-proses  : [10, 20, 30, 40]
Total akhir    : 100

```

Tabel Hasil

Rank	Input	Output (x2)
0	5	10
1	10	20
2	15	30
3	20	40

Jawaban Singkat (Validasi)

Total = 100 karena $(2 \times 5) + (2 \times 10) + (2 \times 15) + (2 \times 20) = 10 + 20 + 30 + 40$.

Tugas Praktikum 5 — Reduce vs Allreduce (MAX & Rata-Rata Global)

Tujuan

- Membedakan Reduce (hasil hanya di root) vs Allreduce (hasil diketahui semua proses).
- Menggunakan MPI.MAX untuk nilai terbesar & MPI.SUM untuk total/rata-rata global.
- Mengagregasi data acak skala besar tanpa mengirim seluruh array.

Spesifikasi

1. **Bagian A (MAX):** Ganti MPI.SUM $\rightarrow$ MPI.MAX. Tentukan proses mana yang punya nilai terbesar.
2. **Bagian B (AVG):** Hitung rata-rata global = (global sum) / (size) menggunakan Allreduce.
3. **Bagian C (Total 1000 bilangan acak/proses):** Gunakan Allreduce untuk menghitung total global.

Starter Code

```
# Nama      : Kamila Zahra R.
# NPM       : 247006111161
# Kelas     : F
# Deskripsi: Tugas Praktikum 4 - Hitung Jumlah dengan Scatter + Gather

from mpi4py import MPI
import numpy as np

def main():
    comm = MPI.COMM_WORLD
    rank = comm.Get_rank()
    size = comm.Get_size()

    # Bagian A: MAX dengan Reduce
    my_val = rank * 7 + 3
    max_val = comm.reduce(my_val, op=MPI.MAX, root=0)
    if rank == 0:
        print(f"[REDUCE-MAX] max = {max_val}")
        print("Proses dengan nilai terbesar biasanya rank terakhir")
```

```

(fungsi naik vs rank).")

# Bagian B: Rata-rata global via Allreduce
val_for_avg = rank + 1
global_sum = comm.allreduce(val_for_avg, op=MPI.SUM)
global_avg = global_sum / size
print(f"[ALLREDUCE-AVG] Rank {rank}: sum={global_sum},
avg={global_avg}")

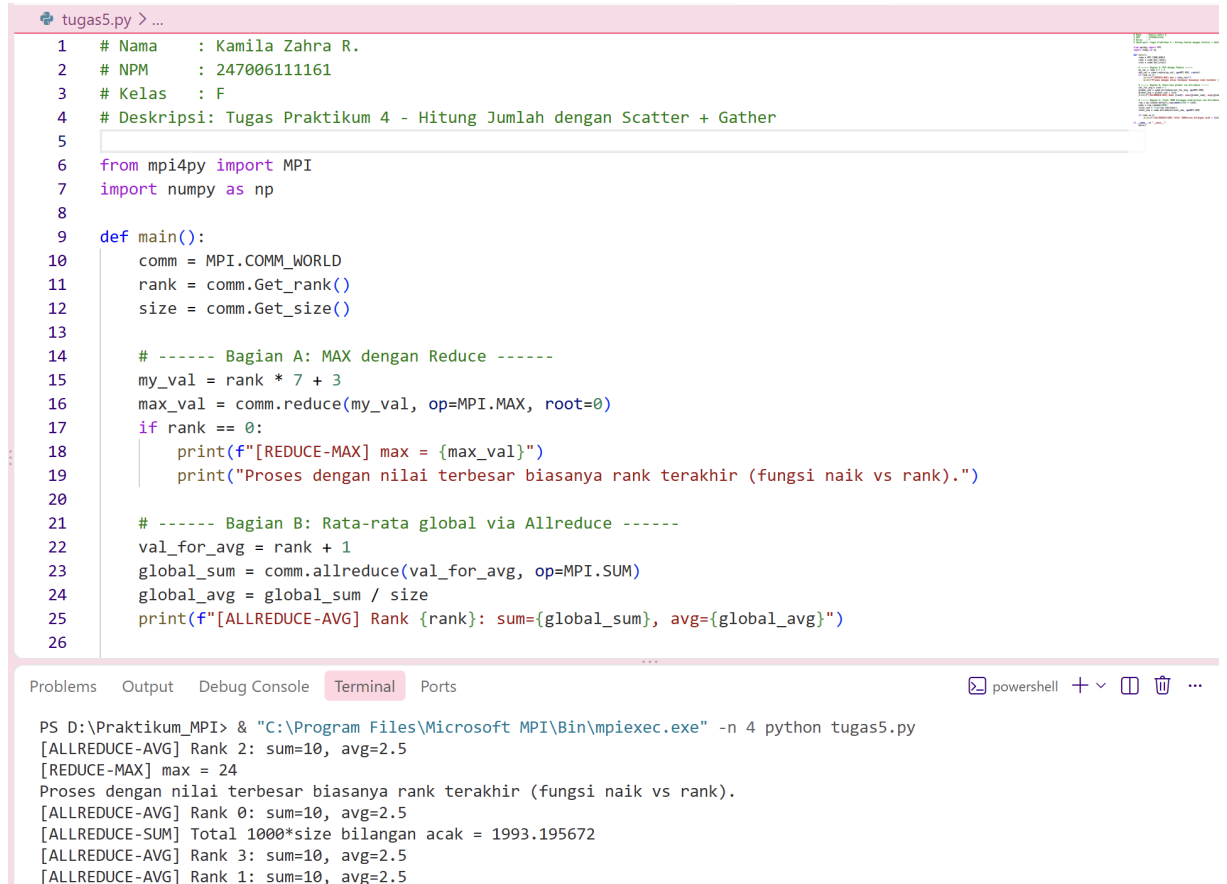
# Bagian C: Total 1000 bilangan acak/proses via Allreduce
rng = np.random.default_rng(seed=12345 + rank)
nums = rng.random(1000)
local_sum = float(np.sum(nums))
total_sum = comm.allreduce(local_sum, op=MPI.SUM)

if rank == 0:
    print(f"[ALLREDUCE-SUM] Total 1000*size bilangan acak =
{total_sum:.6f}")

if __name__ == "__main__":
    main()

```

Tangkapan Layar



```
tugas5.py > ...
1  # Nama      : Kamila Zahra R.
2  # NPM       : 247006111161
3  # Kelas     : F
4  # Deskripsi: Tugas Praktikum 4 - Hitung Jumlah dengan Scatter + Gather
5
6  from mpi4py import MPI
7  import numpy as np
8
9  def main():
10     comm = MPI.COMM_WORLD
11     rank = comm.Get_rank()
12     size = comm.Get_size()
13
14     # ----- Bagian A: MAX dengan Reduce -----
15     my_val = rank * 7 + 3
16     max_val = comm.reduce(my_val, op=MPI.MAX, root=0)
17     if rank == 0:
18         print(f"[REDUCE-MAX] max = {max_val}")
19         print("Proses dengan nilai terbesar biasanya rank terakhir (fungsi naik vs rank).")
20
21     # ----- Bagian B: Rata-rata global via Allreduce -----
22     val_for_avg = rank + 1
23     global_sum = comm.allreduce(val_for_avg, op=MPI.SUM)
24     global_avg = global_sum / size
25     print(f"[ALLREDUCE-AVG] Rank {rank}: sum={global_sum}, avg={global_avg}")
26
```

Problems Output Debug Console **Terminal** Ports

PS D:\Praktikum_MPI> & "C:\Program Files\Microsoft MPI\Bin\mpiexec.exe" -n 4 python tugas5.py
[ALLREDUCE-AVG] Rank 2: sum=10, avg=2.5
[REDUCE-MAX] max = 24
Proses dengan nilai terbesar biasanya rank terakhir (fungsi naik vs rank).
[ALLREDUCE-AVG] Rank 0: sum=10, avg=2.5
[ALLREDUCE-SUM] Total 1000*size bilangan acak = 1993.195672
[ALLREDUCE-AVG] Rank 3: sum=10, avg=2.5
[ALLREDUCE-AVG] Rank 1: sum=10, avg=2.5

Tabel Hasil

Bagian	Metrik	Nilai
A	MAX (Reduce)	24
B	SUM (Allreduce)	10
B	AVG (SUM/Size)	2.5
C	TOTAL (Allreduce)	1993.195672

Pertanyaan Diskusi

1. Jelaskan perbedaan konseptual & praktis Reduce vs Allreduce.

Reduce: Mengagregasi (mengumpulkan dan mengoperasikan) data dari seluruh proses, tetapi hasil akhirnya hanya diserahkan ke satu proses khusus (root process/rank 0). Proses lainnya tidak menerima nilai hasil agregasi tersebut.

Allreduce: Mengagregasi data dari seluruh proses dan membagikan/menyiarkan hasil akhirnya ke SELURUH proses. Praktisnya, semua rank langsung memegang nilai hasil

akhir tanpa perlu dikirim manual oleh root.

2. Mengapa MAX (fungsi naik terhadap rank) memunculkan nilai terbesar di rank terbesar?

Karena nilai lokal dihitung menggunakan rumus matematis $my_val = rank * 7 +$

3. Semakin besar ID Rank-nya, semakin besar pula hasil perkaliannya (fungsi linier naik). Sehingga nilai maksimum ($3 \times 7 + 3 = 24$) otomatis berasal dari rank tertinggi (rank 3).

3. Mengapa Allreduce efisien untuk agregasi besar (mis. 1000 bilangan acak/proses) dibanding mengirim array mentah?

- Efisiensi Bandwidth: Setiap proses hanya menghitung total lokal (*local sum*) dari 1000 datanya di memori masing-masing.
- Yang dikirimkan antar-node melalui jaringan hanyalah 1 nilai float tunggal (hasil jumlah lokal), bukan 1000 data acak mentah. Hal ini sangat menghemat penggunaan jaringan (*network traffic*) dan mengurangi waktu latensi transmisi secara drastis.

PENUTUP

Dengan menyelesaikan kelima praktikum ini, mahasiswa diharapkan: 1. Memahami dasar komunikasi MPI: Scatter, Gather, Reduce, Allreduce. 2. Mampu merancang alur distribusi → komputasi lokal → agregasi hasil. 3. Menyadari implikasi non-determinisme urutan output pada program paralel. 4. Siap mengembangkan studi kasus yang lebih kompleks (prefix-sum paralel, histogram global terdistribusi, dsb).